

How to Terminate a Thread in C# with Examples?

In this article, I am going to discuss **How to Terminate a Thread in C#** with Examples. Please read our previous article where we discussed [Threads Priorities in C#](#) with Examples.

How to Terminate a Thread in C#?

In C#, a thread can be terminated by calling the `Abort()` method of the thread instance. The `Abort()` method throws `ThreadAbortException` to the thread in which it is called. Due to this exception, the thread is terminated.

If the `Abort()` method is called on a thread before calling the `Start()` method on it, calling the `Start()` method on such a thread, later on, will not start it, but it will throw the `ThreadStateException` in the thread which called the `Abort()` method and abort both the threads.

If the `Abort()` method is called on a thread that has started and is in either of the blocked states i.e. Waiting State, Sleep State, or Join State, will first interrupt the thread and then abort it.

There are two overloaded versions of the `Abort()` method available in the `Thread` class. They are as follows:

1. **`Abort()`**: This method raises a `System.Threading.ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread. It will throw `ThreadStateException` if the thread that is being aborted is currently suspended. It will throw `SecurityException` if the caller does not have the required permission.
2. **`Abort(object stateInfo)`**: This method raises a `System.Threading.ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread while also providing exception information about the thread termination. Calling this method usually terminates the thread. Here, the parameter `stateInfo` specifies an object that contains application-specific information, such as the state, which can be used by the thread being aborted. It will throw `ThreadStateException` if the thread that is being aborted is currently suspended. It will throw `SecurityException` if the caller does not have the required permission.

Example to Understand `Abort()` Method of Thread Class in C#:

Let us see an example to Understand the **`Abort()`** Method of Thread Class in C# to terminate a thread. This method raises a `ThreadAbortException` in the thread on which it is invoked, to begin the process of terminating the thread. Generally, this method is used to terminate the thread. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Creating and initializing threads
            Thread thread = new Thread(SomeMethod);
```

```

        thread.Start();
        Console.WriteLine("Thread is Abort");
        // Abort thread Using Abort() method
        thread.Abort();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        for (int x = 0; x < 3; x++)
        {
            Console.WriteLine(x);
        }
    }
}

```

Output: Thread is Abort

The above example shows the use of the Abort() method which is provided by the Thread class. By using thread.Abort(); statement, we can terminate the execution of the thread.

Example to Understand Abort(object stateInfo) Method of Thread Class in C#:

Let us see an example to Understand the **Abort(object stateInfo)** Method of Thread Class in C# to terminate a thread. This method raises a ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread while also providing exception information about the thread termination. Generally, this method is used to terminate the thread. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread thread = new Thread(SomeMethod)
            {
                Name = "Thread 1"
            };
            thread.Start();
            Thread.Sleep(1000);
            Console.WriteLine("Abort Thread Thread 1");
            thread.Abort(100);
            // Waiting for the thread to terminate.
            thread.Join();
            Console.WriteLine("Main thread is terminating");
            Console.ReadKey();
        }
    }
}

```

```

public static void SomeMethod()
{
    try
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} is Starting");
        for (int j = 1; j <= 100; j++)
        {
            Console.Write(j + " ");
            if ((j % 10) == 0)
            {
                Console.WriteLine();
                Thread.Sleep(200);
            }
        }
        Console.WriteLine($"{Thread.CurrentThread.Name} Exiting Normal");
    }
    catch (ThreadAbortException ex)
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} is aborted and the code is {ex.ExceptionState}");
    }
}
}
}

```

Output:

```

Thread 1 is Starting
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
Abort Thread Thread 1
Thread 1 is aborted and the code is 100
Main thread is terminating

```

Calling the Abort() method on a Running Thread in C#:

In the below example, we are calling the Abort() method on the running thread. This will throw the ThreadAbortException and abort the thread on which the Abort() method is called. Calling the Abort() method will throw a ThreadAbortException, so we will enclose its statements within a try-catch block to catch the exception.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Creating an object Thread class
        Thread thread = new Thread(SomeMethod)
        {
            Name = "My Thread1"
        };
        thread.Start();
        //Making the main Thread sleep for 1 second
        //Giving the child thread enough time to start its execution
        Thread.Sleep(1000);
        //Calling the Abort() on thread object
        //This will abort the new thread and throw ThreadAbortException in
        thread.Abort();
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        try
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Started its
Execution");
            for (int i = 0; i < 3; i++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} is printing {i}");
                //Calling the Sleep() method to make it sleep and
                //suspend for 2 seconds after printing a number
                Thread.Sleep(1000);
            }
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
        }
        catch (ThreadAbortException e)
        {
            Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
        }
    }
}
}

```

Output:

```

My Thread1 Has Started its Execution
My Thread1 is printing 0
My Thread1 is printing 1
ThreadAbortException Occurred, Message : Thread was being aborted.

```

Calling the Abort() method on a thread that hasn't started yet.

In the below example, we are calling the Abort() method on a thread before calling the Start() method on it. Calling the Start() method on such a thread, later on, will not start it, but throw the ThreadStartException. For a better understanding, please have a look at the below example.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                //Creating an object Thread class
                Thread MyThread = new Thread(SomeMethod)
                {
                    Name = "My Thread1"
                };
                //Calling the Abort() method on MyThread which hasn't started
                //This will leads to the ThreadStartException
                //And calling the Start() method on the same thread later on v
it and throw ThreadStartException
                MyThread.Abort();
                //Calling the Start() method will not start the thread
                //but throw ThreadStartException and abort it.
                //Because the Abort() method was called on it before it could
                MyThread.Start();
                Console.WriteLine("Main Thread has terminated");
            }
            catch (ThreadStartException e)
            {
                Console.WriteLine($"ThreadStartException Occurred, Message : {
e.Message}");
            }

            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            try
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} Has Started it
Execution");
                for (int i = 0; i < 3; i++)
                {
                    Console.WriteLine($"{Thread.CurrentThread.Name} is printin
                    //Calling the Sleep() method to make it sleep and
```

```

        //suspend for 2 seconds after printing a number
        Thread.Sleep(1000);
    }
    Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
}
catch (ThreadAbortException e)
{
    Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
}
}
}
}
}

```

Output:

```
ThreadStartException Occurred, Message : Thread failed to start.
```

Calling the Abort() method on a Thread that is in Blocked State in C#:

When we call the Abort() method on a thread, which has started its execution, but presently it is in either of the blocked states i.e. Wait State, Sleep State, or Join State, will first interrupt the thread and then abort it by throwing ThreadAbortException. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an object Thread class
            Thread MyThread = new Thread(SomeMethod)
            {
                Name = "My Thread1"
            };
            MyThread.Start();
            //Making the Main thread sleep for 500 milliseconds
            //Which gives enough time for its child start to start its execution
            Thread.Sleep(500);
            //Main thread calling Abort() on the child Thread which is in a blocked state
            //will throw ThreadAbortException
            MyThread.Abort();

            //Main thread has called Join() method on the new thread
            //To wait until its execution is complete
            MyThread.Join();
        }
    }
}

```

```

        Console.WriteLine("Main Thread has terminated");
        Console.ReadKey();
    }
    public static void SomeMethod()
    {
        try
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Started its
Execution");
            for (int i = 0; i < 3; i++)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} is printing {i}");
                //Calling the Sleep() method on newly created thread
                //To make it sleep and suspend for 3 seconds after printing
                Thread.Sleep(3000);
            }
            Console.WriteLine($"{Thread.CurrentThread.Name} Has Finished its
Execution");
        }
        catch (ThreadAbortException e)
        {
            Console.WriteLine($"ThreadAbortException Occurred, Message : {
e.Message}");
        }
    }
}

```

Output:

```

My Thread1 Has Started its Execution
My Thread1 is printing 0
ThreadAbortException Occurred, Message : Thread was being aborted.
Main Thread has terminated

```

In the next article, I am going to discuss [Inter Thread Communication in C#](#) with Examples. Here, in this article, I try to explain **How to Terminate a Thread in C#** with Examples. I hope you enjoy this How to Terminate a Thread in C# with Examples article.